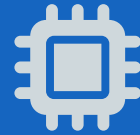


MODELAGEM COMPUTACIONAL



Para Além das Linguagens Regulares

Prof. Dr. José Jairo de S. e Silva

Aula 06

Para Além das Linguagens Regulares

OBJETIVOS DA AULA

🎯 Objetivos da Aula

Até aqui estudamos as **linguagens regulares** - descritas por DFAs, NFAs e expressões regulares (ER). Veremos que há linguagens muito simples que **nenhum autômato finito** reconhece.

Nesta aula vamos:

1. **Lema do Bombeamento (LB)**: ferramenta para provar que certas linguagens **não são regulares**.
2. **Autômatos com Pilha (PDAs)**: modelo mais poderoso, que ganha memória auxiliar.
3. **Gramáticas Livres de Contexto (GLCs)**: formalismo equivalente aos PDAs, com foco em **gerar** strings em vez de reconhecê-las.

Hierarquia de classes de linguagens (não de modelos):

$$\underbrace{L(\text{DFA}) = L(\text{NFA}) = L(\text{ER})}_{\text{Regulares}} \subsetneq \underbrace{L(\text{PDA}) = L(\text{GLC})}_{\text{LLCs}} \subsetneq \underbrace{L(\text{MT})}_{\text{Rec. Enumeráveis}}$$

Os modelos DFA, PDA, MT são **máquinas**; as classes **Regulares**, **LLCs**, **Rec. Enumeráveis** são os conjuntos de linguagens que essas máquinas reconhecem. A inclusão é entre classes.

1. MOTIVAÇÃO: LINGUAGENS NÃO REGULARES

💡 Existem linguagens não regulares?

Uma linguagem L é **regular** se existe algum DFA que a aceita.

Pergunta central: existem linguagens que **nenhum** DFA consegue reconhecer?

A resposta é **sim**. O exemplo clássico:

$$L_{01} = \{0^n 1^n \mid n \geq 1\} = \{01, 0011, 000111, 00001111, \dots\}$$

Essa linguagem exige que o autômato **conte** quantos **0**s foram lidos para depois verificar se há o mesmo número de **1**s. Só que um DFA tem **memória limitada** - seus estados são em número finito e fixo. Contar até um n arbitrário exigiria um número ilimitado de estados.

Por que a intuição não basta? Dizer "não dá para contar" é convincente, mas não é uma prova. Precisamos de um argumento que exclua *qualquer* DFA, não só os que imaginamos. Esse argumento é o **Lema do Bombeamento**.

💡 Zoológico de linguagens não regulares

Linguagens aparentemente simples, mas **todas não regulares**:

Linguagem	Descrição
$\{0^n 1^n \mid n \geq 1\}$	mesmo número de 0s e 1s na ordem $0^* 1^*$
$\{ww^R \mid w \in \{0, 1\}^*\}$	palíndromos de comprimento par
$\{ww \mid w \in \{0, 1\}^*\}$	strings que são uma cópia duplicada de algo
$\{1^p \mid p \text{ é primo}\}$	unários com comprimento primo
$\{a^n b^n c^n \mid n \geq 0\}$	mesma quantidade de a , b e c

O **padrão comum**: todas exigem **memória ilimitada** - contar, comparar posições longínquas ou reconhecer repetições arbitrárias. Autômatos finitos, por definição, não têm essa capacidade.

Veremos que algumas (as três primeiras) passam a ser reconhecidas por **PDAs**. A quarta é mais sutil. A última ($a^n b^n c^n$) é intratável até para PDAs.

2. LEMA DO BOMBEAMENTO

Lema do Bombeamento (LB) - enunciado

Lema 4.1.1 (Lema do Bombeamento): Seja L uma linguagem regular infinita. Então existe uma constante $t \in \mathbb{N}$ tal que $\forall w \in L$ com $|w| \geq t$, é possível escrever

$$w = xyz, \quad \text{com } x, y, z \in \Sigma^*,$$

de modo que **todas as três condições** abaixo valem:

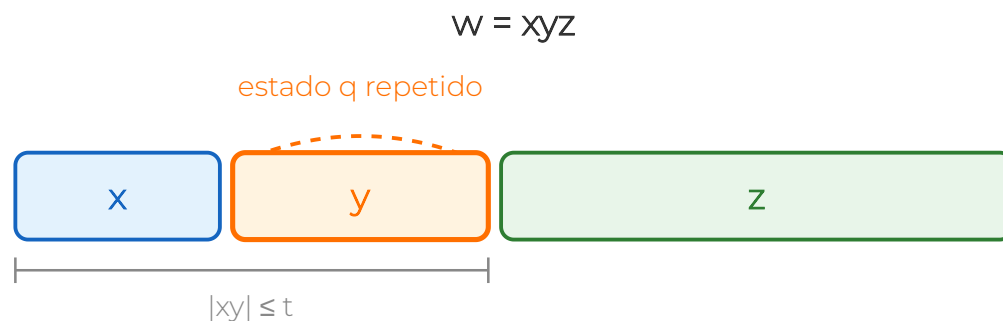
1. $y \neq \varepsilon$ (a parte bombeada não é vazia)
2. $|xy| \leq t$ (a parte bombeada cabe no início)
3. $\forall k \geq 0, xy^kz \in L$ (repetir y não tira w da linguagem)

Em palavras simples: toda string suficientemente longa de uma linguagem regular tem um pedaço y no começo que pode ser "bombeado" - apagado ($k = 0$), mantido ($k = 1$), duplicado ($k = 2$)... - e o resultado continua na linguagem.

⚙️ Por que o LB é verdadeiro?

Ideia da prova: se L é regular, existe um DFA M com t estados que aceita L .

Tome $w \in L$ com $|w| \geq t$. Ao processar w , M visita pelo menos $t + 1$ estados (conta o estado inicial mais um a cada símbolo lido). Como só há t estados, pelo **Princípio da Casa dos Pombos**, algum estado q é visitado **duas vezes** nos primeiros $t + 1$ passos.



y repetida k vezes:

$k=0$: $xy^0z = xz \in L$

$k=1$: $xy^1z = xyz \in L$

$k=2$: $xy^2z = xyxz \in L$

$k \geq 0$: $xy^k z \in L$

A parte x leva do estado inicial até q . A parte y forma um **ciclo** em q (por isso pode ser repetida qualquer número de vezes). A parte z vai de q até um estado final. Percorrer o ciclo mais ou menos vezes ainda leva ao mesmo estado final, então todas as strings xy^kz também são aceitas.

⚙️ Como usar o LB para provar **não** regularidade

O LB é o nosso **contrapositivo**: se L é regular, então vale o LB. Logo, se mostrarmos que o LB **não vale** para L , concluímos que L não é regular.

Receita em 6 passos (prova por contradição):

1. Suponha, por contradição, que L é regular.
2. O LB dá uma constante t .
3. Escolha estrategicamente uma string $w \in L$ com $|w| \geq t$.
4. Pelo LB, existe uma decomposição $w = xyz$ com $y \neq \varepsilon$ e $|xy| \leq t$.
5. Analise **todas as formas possíveis** de y (as condições do LB geralmente restringem bastante).
6. Encontre algum $k \geq 0$ tal que $xy^kz \notin L$. **Contradição!**

Atenção pedagógica: o LB é uma condição **necessária**, mas **não suficiente**. Existem linguagens não regulares que satisfazem o LB. Ou seja, o LB só serve para **provar que uma linguagem NÃO é regular**. Ele nunca prova o contrário.

 Teorema 4.1.2: $L_{01} = \{0^i 1^i\}$ não é regular

Estratégia (informal): escolheremos $w = 0^t 1^t$. Como $|xy| \leq t$, y fica inteiramente nos 0s. Bombear y descompensa a contagem.

Prova formal. Suponha que L_{01} é regular. Pelo LB, seja t a constante. Tome $w = 0^t 1^t \in L_{01}$. Note $|w| = 2t \geq t$.

Pelo LB, $w = xyz$ com $y \neq \varepsilon$ e $|xy| \leq t$. Como os primeiros t símbolos de w são todos 0, a parte xy está **inteiramente dentro do bloco de 0s**. Logo $y = 0^j$ para algum $j \geq 1$.

Pela condição (3), tomando $k = 0$: $xy^0z = xz \in L_{01}$. Mas xz tem $t - j$ zeros e t uns, com $j \geq 1$. Portanto xz tem **menos zeros do que uns**, ou seja, $xz \notin L_{01}$.

Contradição. Logo L_{01} não é regular. $\square$

Ideia central da prova: fixamos w tal que a restrição $|xy| \leq t$ nos **força** a colocar y num bloco homogêneo, e então o bombeamento quebra a igualdade de contagem.

 Teorema 4.1.3: $L_p = \{1^p \mid p \text{ primo}\}$ não é regular

Estratégia: se conseguirmos bombear y de modo a produzir uma string de comprimento **composto**, ela cai fora de L_p .

Prova. Suponha L_p regular; seja t a constante do LB. Escolha um primo $p \geq t$ e tome $w = 1^p$.

Pelo LB, $w = xyz$ com $y = 1^n$ ($n \geq 1$) e $|xz| = p - n$.

Pela condição (3), $xy^{p+1}z \in L_p$. Calculamos:

$$|xy^{p+1}z| = |xz| + (p+1)|y| = (p-n) + (p+1)n = p + np = p(1+n).$$

Como $p \geq 2$ e $1+n \geq 2$, temos $p(1+n)$ sendo o **produto de dois fatores, ambos ≥ 2** . Logo $p(1+n)$ é **composto**, não primo.

Isso contradiz $xy^{p+1}z \in L_p$. Logo L_p não é regular. $\square$

Ideia central: em vez de descompensar contagens (como em L_{01}), aqui manipulamos **propriedades aritméticas** do comprimento. Escolhemos k de modo que o comprimento resultante seja composto.

🔗 Leitura guiada: como ler uma prova com LB

Quando você ler (ou escrever) uma prova com LB, preste atenção em **três decisões estratégicas**:

- 1. Escolha de w .** w é **sua** escolha - você decide qual string atacar. Boas escolhas forçam a estrutura de y . Em L_{01} , escolhemos $w = 0^t 1^t$ porque a restrição $|xy| \leq t$ obriga y a ser feita só de 0s.
- 2. Análise de y .** Depois da escolha de w , as condições (1) e (2) do LB restringem drasticamente y . Normalmente você descobre que y só pode ter uma forma.
- 3. Escolha de k .** Pequenos valores como $k = 0$ (apagar y) ou $k = 2$ (duplicar) costumam bastar. Para linguagens "aritméticas" (primos, quadrados, etc.), valores maiores dependendo da conta.

Pegadinhas frequentes:

- **Não** é você que escolhe x, y, z - o LB garante que *existe alguma* decomposição. Você tem que argumentar **para toda** decomposição possível.
- y pode ser vazio? **Não** - condição (1).
- y pode aparecer no final? **Não necessariamente** - $|xy| \leq t$ força y a estar no início.

❓ Pratique: aplicando o LB

Use o LB para provar que as linguagens abaixo não são regulares:

1. $L_{EQ} = \{w \in \{0, 1\}^* \mid w \text{ tem o mesmo número de 0s e 1s}\}$
2. $L_{RR} = \{ww^R \mid w \in \{0, 1\}^*\}$ (palíndromos de comprimento par)
3. $L_{>} = \{0^i 1^j \mid i > j\}$

Dicas estratégicas:

- Item 1: escolha $w = 0^t 1^t$ e bombeie y (que só pode ter 0s).
- Item 2: escolha $w = 0^t 1^t 1^t 0^t$ e analise xy^0z .
- Item 3: escolha $w = 0^{t+1} 1^t$ e analise xy^0z .

3. AUTÔMATOS COM PILHA (PDAs)

💡 Do autômato finito ao autômato com pilha

Já provamos que $L_{01} = \{0^n 1^n\}$ não é regular: nenhum DFA tem estados suficientes para contar um n arbitrário. Precisamos de um modelo com **memória auxiliar**.

Ideia-chave: acoplar ao autômato uma **pilha** (estrutura LIFO). A pilha é ilimitada, mas o acesso é **restrito**: o autômato só enxerga o símbolo do **topo**; não pode olhar posições arbitrárias.



Por que a pilha muda tudo? A pilha permite **lembrar quantos** símbolos foram empilhados e **comparar** com os próximos. Para L_{01} : ao ler n zeros, empilhamos n marcadores; ao ler os 1 s, desempilhamos um a cada 1 . Se a pilha esvaziar exatamente quando a entrada terminar, a string é aceita.

Definição 4.2.1: Autômato com Pilha (PDA)

Um **PDA** P é uma **7-tupla** $P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$ onde:

Componente	Significado
Q	conjunto finito de estados
Σ	alfabeto de entrada
Γ	alfabeto da pilha (pode conter símbolos fora de Σ)
δ	função de transição (ver abaixo)
$q_0 \in Q$	estado inicial
$Z_0 \in \Gamma$	símbolo inicial da pilha (marca o fundo)
$F \subseteq Q$	estados finais

A função de transição tem a seguinte assinatura:

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \longrightarrow \mathcal{P}(Q \times \Gamma^*)$$

onde $\mathcal{P}(\cdot)$ é o conjunto das partes (todos os subconjuntos finitos).

☰ Em palavras simples: o que δ representa

$\delta(q, a, X)$ responde à pergunta:

|"Estando em q , lendo a na entrada, com X no topo da pilha - **quais** são as escolhas possíveis?"

Cada escolha é um par (p, α) onde:

- p é o **novo estado**
- $\alpha \in \Gamma^*$ é a **sequência a empilhar** (no lugar de X , que foi desempilhado)

O conjunto $\mathcal{P}(Q \times \Gamma^*)$ codifica o **não determinismo**: para cada tripla (q, a, X) , o PDA pode ter **zero, uma ou várias** transições possíveis. O PDA aceita se **alguma** dessas escolhas leva à aceitação.

Diferenças em relação ao NFA:

- A transição depende também do **símbolo no topo da pilha**
- A transição **empilha** uma sequência de símbolos de volta
- Transições por ϵ (sem consumir entrada) continuam disponíveis

|" **Resumo:** um PDA é um NFA que, além de mudar de estado, consulta o topo da pilha e atualiza o topo.

⚙️ Como o PDA computa: passo a passo

A cada passo, o PDA executa **quatro ações** simultâneas:

1. **Consome** um símbolo da entrada (ou ϵ , sem avançar)
2. **Desempilha** o símbolo do topo (obrigatoriamente)
3. **Muda** para um novo estado (possivelmente o mesmo)
4. **Empilha** uma sequência α de símbolos (pode ser vazia)

Notação $a, X/\alpha$: rótulo padrão das transições em diagramas de PDA.

Lendo a da entrada com X no topo da pilha: muda de estado, desempilha X e empilha α . **Convenção (HMU):** o **primeiro** símbolo de α é o que fica no **topo** após empilhar.

Exemplo: a transição $0, Z_0/0 Z_0$ significa:

- Lê o símbolo 0 da entrada
- Desempilha Z_0 do topo
- Empilha a string $0 Z_0$: 0 fica no topo e Z_0 logo abaixo

Efeito líquido: empilhar 0 preservando Z_0 no fundo.

Exemplo formal: o PDA do simulador

O simulador da aula implementa o seguinte PDA para

$$L = \{0^n 1^n \mid n \geq 1\}$$

com aceitação por **estado final**:

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

onde

- $Q = \{q_0, q_1, q_2\}$
- $\Sigma = \{0, 1\}$
- $\Gamma = \{0, Z_0\}$
- $F = \{q_2\}$

Leitura rápida: este é o PDA mais simples para $\{0^n 1^n\}$ com aceitação por estado final. Em q_0 ele empilha; em q_1 ele desempilha; em q_2 ele aceita.

 Tabela de transições do PDA do simulador

As transições formais são:

Transição	Leitura
$\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$	leu o primeiro 0 , empilha marcador
$\delta(q_0, 0, 0) = \{(q_0, 00)\}$	continua empilhando um 0 por símbolo
$\delta(q_0, 1, 0) = \{(q_1, \epsilon)\}$	ao ver o primeiro 1 , muda para a fase de desempilhar
$\delta(q_1, 1, 0) = \{(q_1, \epsilon)\}$	cada 1 remove um marcador
$\delta(q_1, \epsilon, Z_0) = \{(q_2, Z_0)\}$	quando sobra só o fundo, aceita

 **Ideia do autômato:** em q_0 a pilha conta quantos **0**s apareceram; em q_1 os **1**s “gastam” essa contagem.

🔍 Exemplo destrinchado: como ler a 7-tupla

No exemplo anterior,

$$P = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$$

significa concretamente:

- **Estados:** q_0 = empilhar, q_1 = casar com os 1s, q_2 = aceitar
- **Entrada** $\Sigma = \{0, 1\}$: o autômato só lê zeros e uns
- **Pilha** $\Gamma = \{0, Z_0\}$: usa um **marcador de fundo** Z_0 e o símbolo **0** como marcador de contagem
- **Estado inicial** q_0 : toda computação começa empilhando
- **Símbolo inicial da pilha** Z_0 : a pilha nunca começa vazia; ela começa marcada com o fundo
- **Estados finais** $F = \{q_2\}$: aceitar significa conseguir chegar a q_2

Leitura importante de δ :

$$\delta(q_0, 0, Z_0) = \{(q_0, 0Z_0)\}$$

Quer dizer:

1. o PDA está em q_0
2. lê um **0** da entrada
3. encontra Z_0 no topo
4. troca Z_0 por **0** Z_0
5. continua em q_0

Efeito líquido: a pilha passa de $[Z_0]$ para $[0, Z_0]$. Em outras palavras: “vi um zero, então guardei um marcador”.

▶ Simulador interativo: PDA para $L = \{0^n 1^n\}$

Use o widget para ver um PDA funcionando sobre a linguagem $L = \{0^n 1^n\}$. Ele separa a computação em duas fases:

- q_0 : lê 0s e empilha marcadores
- q_1 : lê 1s e desempilha

Se a entrada termina exatamente quando a pilha volta a Z_0 , a palavra é aceita.

Simulador de Pilha (PDA): $L = \{0^n 1^n\}$

Exemplos para testar: 01, 0011, 000111, 001, 0101.

Ex.: 0011

Simular

Fita de entrada

Estado atual: q_0 | Cabeça: início da fita

cabeça



Log da computação

📁 Variante com Z_0 e Z_1 explícitos

Para deixar claro que o **alfabeto da pilha** pode ser diferente do alfabeto de entrada, podemos usar outro PDA equivalente:

$$P' = (\{q_0, q_1, q_2\}, \{0, 1\}, \{Z_0, Z_1\}, \delta, q_0, Z_0, \{q_2\})$$

com transições

$$\delta(q_0, 0, Z_0) = \{(q_0, Z_1 Z_0)\} \quad \delta(q_0, 0, Z_1) = \{(q_0, Z_1 Z_1)\}$$

$$\delta(q_0, 1, Z_1) = \{(q_1, \varepsilon)\} \quad \delta(q_1, 1, Z_1) = \{(q_1, \varepsilon)\}$$

$$\delta(q_1, \varepsilon, Z_0) = \{(q_2, Z_0)\}$$

Aqui:

- Z_0 = **fundo da pilha**
- Z_1 = **marcador de contagem**

Leitura pedagógica: agora a entrada continua sendo $\{0, 1\}$, mas a pilha usa um alfabeto diferente. Isso deixa mais visível o papel de cada símbolo.

 Execução com Z_0 e Z_1 : palavra 0011

Execução de exemplo para 0011:

Passo	ID	Interpretação
0	$(q_0, 0011, Z_0)$	início
1	$(q_0, 011, Z_1Z_0)$	leu o primeiro 0, empilhou Z_1
2	$(q_0, 11, Z_1Z_1Z_0)$	leu o segundo 0, empilhou outro Z_1
3	$(q_1, 1, Z_1Z_0)$	leu o primeiro 1, removeu um Z_1
4	(q_1, ϵ, Z_0)	leu o segundo 1, removeu o outro Z_1
5	(q_2, ϵ, Z_0)	como sobrou só Z_0 , aceita

 **Moral:** Z_0 não “conta” nada; ele só marca o fundo. Quem conta é o símbolo escolhido para marcar repetições, aqui Z_1 .

☰ Definição formal: linguagem de um PDA

Descrição instantânea (ID): uma tripla (q, w, γ) onde $q \in Q$ (estado atual), $w \in \Sigma^*$ (entrada não lida), $\gamma \in \Gamma^*$ (pilha).

Passo computacional $\vdash_P$: se $(p, \alpha) \in \delta(q, a, X)$, então para todo $w \in \Sigma^*$ e $\beta \in \Gamma^*$:

$$(q, aw, X\beta) \vdash_P (p, w, \alpha\beta)$$

$\vdash_P^*$ representa **zero ou mais passos** (fecho transitivo reflexivo).

Linguagem do PDA (Definição 4.2.5):

$$L(P) = \{w \in \Sigma^* \mid (q_0, w, Z_0) \vdash_P^* (q, \varepsilon, \alpha), \text{ com } q \in F, \alpha \in \Gamma^*\}$$

Em palavras: P aceita w se **existe** alguma sequência de escolhas que esgota a entrada e termina em um **estado final** (independente do conteúdo da pilha).

☰ Dois critérios de aceitação

Um PDA admite **duas convenções equivalentes** de aceitação:

Critério	Aceita w quando...	Notação
Estado final	termina a entrada em um estado $q \in F$	$L(P)$
Pilha vazia	termina a entrada com a pilha completamente vazia	$N(P)$

Teoremas 4.2.1 e 4.2.2 (equivalência):

- Para todo PDA P existe um PDA P' com $N(P') = L(P)$
- Para todo PDA P existe um PDA P' com $L(P') = N(P)$

Assim, a classe $\{L(P)\}$ coincide com $\{N(P)\}$: escolhemos a convenção conveniente para cada problema.

Nos exemplos desta aula usaremos **aceitação por estado final**. No PDA P_{RR} , quando q_2 é alcançado, Z_0 ainda permanece na pilha - isso é **compatível** com a convenção, pois a aceitação olha apenas para o estado, não para a pilha.

 PDA para $L_{RR} = \{ww^R\}$: estratégia

L_{RR} contém palíndromos de comprimento par: ε , 00, 11, 0110, 1001, ...

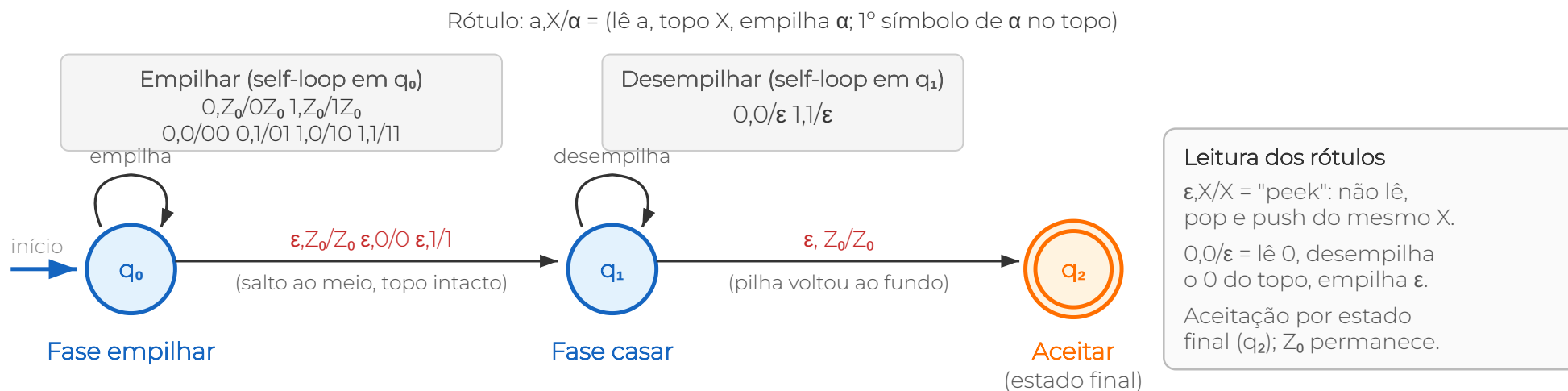
Intuição: para aceitar ww^R , o PDA precisa armazenar a primeira metade w e comparar, símbolo a símbolo, com a segunda metade (que é w invertida). A pilha é ideal porque devolve os símbolos na **ordem inversa** em que foram empilhados.

Estratégia do PDA (quatro passos):

1. **Fase de empilhamento** (estado q_0): empilha cada símbolo lido
2. **Salto ao meio** ($q_0 \rightarrow q_1$ por ε): decide não deterministicamente que chegou ao meio da string
3. **Fase de casamento** (estado q_1): para cada símbolo lido, desempilha o topo e verifica igualdade
4. **Aceitação** ($q_1 \rightarrow q_2$ por ε): ocorre quando a pilha voltou a Z_0 (só o marcador de fundo)

Por que o salto é não determinístico? O PDA não sabe onde termina a primeira metade. Ele tenta todos os pontos possíveis; se **alguma** escolha leva à aceitação, a string pertence à linguagem. Essa habilidade de "adivinhar" é o que torna PDAs estritamente mais poderosos que DPDAs.

PDA para L_{RR} : diagrama



Como ler o diagrama (parte 1):

1. **Self-loop em q_0 (empilhar).** Para cada par (símbolo lido, símbolo no topo), há um rótulo que empilha o símbolo lido sobre o topo. Ex.: **0, 1/01** significa "leu **0** com **1** no topo, empilha **01** (o **0** fica no topo)".
2. **Setas vermelhas (ϵ -transições).** Não consomem entrada. A de $q_0 \rightarrow q_1$ é o **chute não determinístico** "cheguei ao meio"; preserva o topo ($\epsilon, X/X$). A de $q_1 \rightarrow q_2$ só dispara quando o topo é Z_0 .

PDA para L_{RR} : diagrama (leitura, parte 2)

Como ler o diagrama (parte 2):

1. **Self-loop em q_1 (casar)**. O símbolo lido precisa bater com o topo: $0, 0/\epsilon$ ou $1, 1/\epsilon$. Se não bater, esse ramo do não determinismo morre.
2. **Aceitação**. Chegar em q_2 (círculo duplo) com a entrada esgotada. Z_0 permanece na pilha, mas a convenção é **estado final**: olha só o estado.

Moral: o PDA tenta **todos** os pontos possíveis de "meio". Aceita se **alguma** escolha consegue esvaziar a pilha até Z_0 no mesmo instante em que a entrada acaba.

 Simulando o PDA: string $w = 0110$

Usamos a **descrição instantânea** (ID) (q , entrada restante, pilha). Convenção: topo da pilha à esquerda.

Passo	Estado	Entrada restante	Pilha	Ação
0	q_0	0110	Z_0	início
1	q_0	110	$0Z_0$	lê 0, empilha 0
2	q_0	10	$10Z_0$	lê 1, empilha 1
3	q_1	10	$10Z_0$	ϵ -transição $q_0 \rightarrow q_1$
4	q_1	0	$0Z_0$	lê 1, desempilha 1 (match)
5	q_1	ϵ	Z_0	lê 0, desempilha 0 (match)
6	q_2	ϵ	Z_0	ϵ -transição para estado final

$w = 0110$ é aceita ✓. O PDA "adivinhou" corretamente que o meio fica entre os passos 2 e 3.

☰ PDAs determinísticos (DPDAs)

Um PDA é **determinístico** (DPDA) quando δ satisfaz:

1. $|\delta(q, a, X)| \leq 1$ para todo (q, a, X) : **no máximo uma transição** por configuração
2. Se $\delta(q, a, X) \neq \emptyset$ para algum $a \in \Sigma$, então $\delta(q, \varepsilon, X) = \emptyset$: **sem conflito** entre ler e fazer ε -transição

Atenção: ao contrário de DFAs e NFAs (equivalentes), **DPDAs são estritamente menos poderosos** que PDAs não determinísticos. Existem LLCs que nenhum DPDA aceita - por exemplo, $L_{RR} = \{ww^R\}$, porque o DPDA não pode "adivinhar" o meio da string.

Consequência prática: parsers de linguagens de programação precisam de análise eficiente e determinística. Por isso usam subclasses de GLCs compatíveis com DPDAs: **LL(k)**, **LR(k)**, **LALR**.

4. GRAMÁTICAS LIVRES DE CONTEXTO

💡 Reconhecer versus gerar

PDA's e GLCs descrevem **a mesma classe de linguagens** (as LLCs), mas sob perspectivas **opostas**:

- Um **PDA** é uma máquina que **reconhece** strings: dado w , responde aceita/rejeita.
- Uma **GLC** é um sistema de regras que **gera** strings: produz, uma a uma, as w da linguagem.

Analogia com português: podemos descrever a estrutura sintática de uma frase:

- Frase $\rightarrow$ Sujeito Predicado
- Sujeito $\rightarrow$ O gato | O cachorro
- Predicado $\rightarrow$ comeu o rato | fugiu

Frase $\Rightarrow$ Sujeito Predicado $\Rightarrow$ O gato Predicado $\Rightarrow$ O gato comeu o rato

Nas GLCs fazemos o mesmo formalmente: variáveis (letras maiúsculas) = categorias; terminais = símbolos do alfabeto final; regras = como expandir cada variável.

Definição 4.3.1: GLC

Uma **Gramática Livre de Contexto** é uma **quádrupla** $G = (V, T, P, S)$ onde:

Componente	Significado
V	conjunto finito de variáveis (não terminais)
T	conjunto finito de terminais , com $V \cap T = \emptyset$
P	conjunto de regras de produção $X \rightarrow W$, com $X \in V$ e $W \in (V \cup T)^*$
$S \in V$	variável inicial

Em palavras simples:

- As **variáveis** são nomes abstratos para "categorias sintáticas" (ex.: E = expressão, I = identificador).
- Os **terminais** são os símbolos que aparecem nas strings finais (ex.: $0, 1, +, *$).
- As **regras** dizem como substituir uma variável por uma string de variáveis e terminais.
- A variável S é por onde **começamos** a gerar qualquer string.

Notação compacta: regras com mesma variável à esquerda são agrupadas com $|$. Ex.:
 $X \rightarrow aX \mid bX \mid \varepsilon$.

Exemplo 4.3: GLC para palíndromos binários

A gramática $G_{\text{pal}} = (\{S\}, \{0, 1\}, P, S)$ com regras:

$$S \rightarrow \varepsilon \mid 0 \mid 1 \mid 0S0 \mid 1S1$$

Leitura das regras:

- $S \rightarrow \varepsilon$: palíndromo vazio (caso base)
- $S \rightarrow 0 \mid 1$: palíndromo de um único símbolo (caso base)
- $S \rightarrow 0S0$: "envelopa" um palíndromo menor com 0s nas extremidades
- $S \rightarrow 1S1$: idem com 1s

Por que isso importa: apenas **5 regras compactas** descrevem a linguagem dos palíndromos binários - linguagem que, pelo Exercício 4.1, **nenhum DFA** reconhece. GLCs são dramaticamente mais concisas que DFAs para certas linguagens.

Exemplo 4.3: derivações passo a passo

Cada $\Rightarrow$ aplica uma regra, substituindo uma variável na forma sentencial:

String gerada	Derivação	Regras aplicadas
00	$S \Rightarrow 0S0 \Rightarrow 0\varepsilon 0 = 00$	$S \rightarrow 0S0; S \rightarrow \varepsilon$
0110	$S \Rightarrow 0S0 \Rightarrow 0(1S1)0 \Rightarrow 0(1\varepsilon 1)0 = 0110$	$S \rightarrow 0S0; S \rightarrow 1S1; S \rightarrow \varepsilon$
1001	$S \Rightarrow 1S1 \Rightarrow 1(0S0)1 \Rightarrow 1(0\varepsilon 0)1 = 1001$	$S \rightarrow 1S1; S \rightarrow 0S0; S \rightarrow \varepsilon$
10101	$S \Rightarrow 1S1 \Rightarrow 1(0S0)1 \Rightarrow 1(010)1 = 10101$	$S \rightarrow 1S1; S \rightarrow 0S0; S \rightarrow 1$

Conclusão: $L(G_{\text{pal}}) = \{w \in \{0, 1\}^* \mid w = w^R\}$, o conjunto de **todos** os palíndromos binários.

 Exemplo: GLC para $L_{01} = \{0^n 1^n\}$

Retomando o exemplo que motivou toda a aula: L_{01} **não é regular**, mas é **LLC**.

A gramática $G_{01} = (\{S\}, \{0, 1\}, P, S)$ com regras:

$$S \rightarrow 0S1 \mid \varepsilon$$

Derivações:

String	Derivação
ε	$S \Rightarrow \varepsilon$
01	$S \Rightarrow 0S1 \Rightarrow 0\varepsilon 1 = 01$
0011	$S \Rightarrow 0S1 \Rightarrow 0(0S1)1 \Rightarrow 00\varepsilon 11 = 0011$
000111	$S \Rightarrow 0S1 \Rightarrow 00S11 \Rightarrow 000S111 \Rightarrow 000\varepsilon 111$

Em palavras simples: cada aplicação de $S \rightarrow 0S1$ adiciona **um 0 à esquerda e um 1 à direita simultaneamente** - por construção, a contagem fica sempre balanceada. $S \rightarrow \varepsilon$ finaliza a derivação. Comparado a tentar escrever um DFA (impossível), a GLC tem **duas regras**.

Exemplo 4.4: GLC para expressões aritméticas

$G_1 = (\{E, I\}, \{a, b, 0, 1, +, *, (,)\}, P, E)$ com regras:

Expressões (E):

$$E \rightarrow I \mid E + E \mid E * E \mid (E)$$

Identificadores (I):

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$$

Derivação de $a * (a + b00)$:

$$E \Rightarrow E * E \Rightarrow I * E \Rightarrow a * E$$

$$\Rightarrow a * (E) \Rightarrow a * (E + E)$$

$$\Rightarrow a * (I + E) \Rightarrow a * (a + E)$$

$$\Rightarrow a * (a + I) \Rightarrow a * (a + I0)$$

$$\Rightarrow a * (a + I00) \Rightarrow a * (a + b00)$$

Leitura: E gera qualquer expressão; I gera qualquer identificador (letra a ou b opcionalmente seguida de $a, b, 0, 1$). É o tipo de gramática usada no **front-end de compiladores** para descrever expressões.

Derivações: definição formal

Definição 4.3.2: Dada $G = (V, T, P, S)$, escrevemos $\alpha A \beta \Rightarrow_G \alpha \gamma \beta$ se $A \rightarrow \gamma$ é regra de P . (Substituímos A por γ em alguma posição.)

Definição 4.3.3: $\Rightarrow_G^*$ ("zero ou mais passos") é definido indutivamente:

- **Base:** $\alpha \Rightarrow_G^* \alpha$ para todo α
- **Indução:** se $\alpha \Rightarrow_G^* \beta$ e $\beta \Rightarrow_G \gamma$, então $\alpha \Rightarrow_G^* \gamma$

Definição 4.3.4 (Linguagem de G):

$$L(G) = \{w \in T^* \mid S \Rightarrow_G^* w\}$$

$L(G)$ é o conjunto de strings de **terminais** que se obtêm partindo de S .

Terminologia:

- **Forma sentencial:** qualquer string derivável (pode conter variáveis).
- **String terminal:** forma sentencial feita apenas de terminais - é o alvo final.

🌳 Árvores de derivação (Parse Trees)

Uma **árvore de derivação** representa graficamente uma derivação: mostra **qual regra foi aplicada em cada passo** e **como as variáveis se expandiram**.

Regras para construir a árvore:

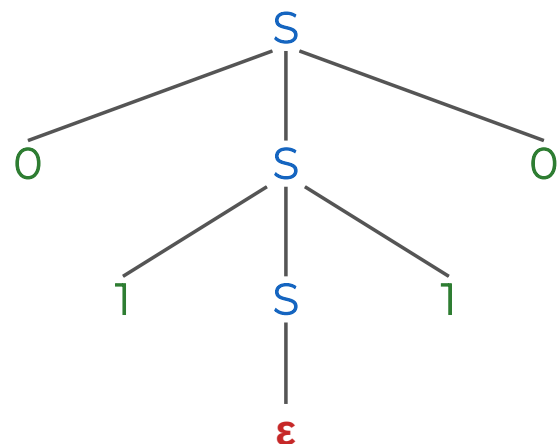
- A **raiz** é a variável inicial S
- Cada **nó interno** é uma variável
- Cada **folha** é um terminal ou ϵ
- Se a regra $A \rightarrow X_1 X_2 \dots X_k$ é aplicada em um nó A , então ele tem filhos $X_1, X_2, \dots, X_k$, da esquerda para a direita

Leitura da string (yield da árvore): concatene as folhas da esquerda para a direita, omitindo ϵ .

Por que árvores importam: uma árvore codifica **estrutura** (a hierarquia das aplicações de regras), não apenas a sequência de regras. Árvores serão essenciais para discutir **ambiguidade** e **precedência** de operadores.

Árvore de derivação de 0110

Usando G_{pal} (variável S): aplicamos $S \rightarrow 0S0$, depois $S \rightarrow 1S1$, e por fim $S \rightarrow \epsilon$.



A string 0110 é gerada por esta árvore

Regras aplicadas:

1º: $S \rightarrow 0S0$

2º: $S \rightarrow 1S1$

3º: $S \rightarrow \epsilon$

Folhas (esq. → dir.):

$0 \cdot 1 \cdot \epsilon \cdot 1 \cdot 0 = 0110$

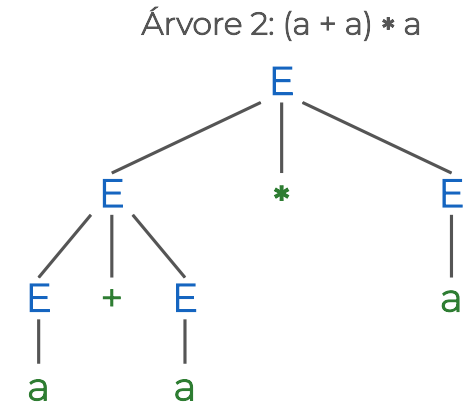
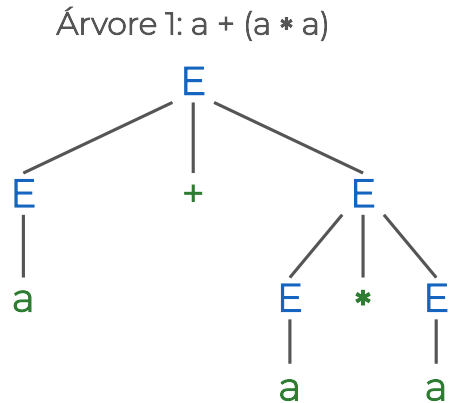
⚠ Ambiguidade de gramáticas

Definição 4.3.5: Uma gramática G é **ambígua** se existe alguma string $w \in L(G)$ que possui duas ou mais **árvores de derivação diferentes**.

Em palavras simples: ambiguidade significa que a gramática permite "analisar" a mesma string de formas estruturalmente distintas. Como a estrutura captura **significado** (precedência, associação, interpretação), ambiguidade é uma ameaça direta à interpretação única.

Exemplo clássico: a gramática G_1 do Exemplo 4.4 é ambígua. A string $a + a * a$ tem duas árvores de derivação, cada uma correspondendo a uma interpretação diferente: $a + (a * a)$ ou $(a + a) * a$.

⚠ Ambiguidade: duas árvores para $a + a * a$



Folhas: $a + a * a$ - interpretação: $a + (a * a)$

Folhas: $a + a * a$ - interpretação: $(a + a) * a$

Impacto: duas árvores → duas interpretações → dois resultados numéricos diferentes. Gramáticas para linguagens de programação **precisam** ser não ambíguas para que o compilador gere código único.

Eliminando ambiguidade: precedência de operadores

G_1 é ambígua porque não codifica que $*$ tem **precedência** sobre $+$. Para eliminar, **estratificamos** a gramática em níveis.

Gramática não ambígua (três níveis: E, T, F):

$$E \rightarrow T \mid E + T$$

$$T \rightarrow F \mid T * F$$

$$F \rightarrow I \mid (E)$$

$$I \rightarrow a \mid b \mid Ia \mid Ib \mid I0 \mid I1$$

Papel de cada variável:

- E gera **somas** de termos ($T + T + \dots$)
- T gera **produtos** de fatores ($F * F * \dots$)
- F gera um identificador ou uma expressão entre parênteses

Como T só aparece **abaixo** de E , e F só **abaixo** de T , a árvore **sempre** coloca $*$ abaixo de $+$ - ou seja, $*$ é avaliado primeiro. A precedência está **codificada na estrutura** da gramática, não é uma regra externa.

Derivação única de $a + a * a$

Aplicando a gramática estratificada:

Passo	Forma sentencial	Regra aplicada
0	E	(início)
1	$E + T$	$E \rightarrow E + T$
2	$T + T$	$E \rightarrow T$
3	$F + T$	$T \rightarrow F$
4	$I + T$	$F \rightarrow I$
5	$a + T$	$I \rightarrow a$
6	$a + T * F$	$T \rightarrow T * F$
7	$a + F * F$	$T \rightarrow F$
8	$a + a * a$	aplicações de $F \rightarrow I$ e $I \rightarrow a$

Conclusão: a multiplicação fica sempre **mais próxima das folhas** que a adição, garantindo a interpretação $a + (a * a)$. Essa é **a única árvore possível** na gramática estratificada.

5. EQUIVALÊNCIA PDA $\leftrightarrow$ GLC

↔ Teorema 4.3.1: PDAs e GLCs são equivalentes

Teorema 4.3.1: Uma linguagem L é gerada por alguma GLC **se e somente se** existe algum PDA que a aceita.

Ou seja, PDAs e GLCs descrevem **exatamente a mesma classe** de linguagens: as **Linguagens Livres de Contexto (LLCs)**.

$$L(\text{PDA}) = L(\text{GLC}) = \text{LLCs}$$

Duas direções da prova:

1. **GLC $\Rightarrow$ PDA:** dada G , construir um PDA que **simula derivações** usando a pilha para guardar a forma sentencial corrente.
2. **PDA $\Rightarrow$ GLC:** dado P , construir G cujas regras codificam "quando o PDA consome uma entrada e retorna a pilha a um nível anterior".

Leitura guiada: o teorema explica por que temos dois formalismos tão diferentes (máquinas vs regras) para a mesma classe. Cada um é mais natural para certos problemas: PDA para **reconhecer**; GLC para **gerar** e **explicar a estrutura**.

Propriedades: regulares vs LLCs

Propriedade	Regulares	LLCs
Fechadas sob união $L_1 \cup L_2$	✓	✓
Fechadas sob concatenação $L_1 \cdot L_2$	✓	✓
Fechadas sob estrela L^*	✓	✓
Fechadas sob interseção $L_1 \cap L_2$	✓	✗
Fechadas sob complemento $\overline{L}$	✓	✗

Leitura da tabela: LLCs **herdam** união, concatenação e estrela das regulares, mas **perdem** o fechamento sob interseção e complemento. É uma diferença estrutural importante - a classe é mais expressiva, mas menos bem-comportada algebricamente.

☒ LLCs não fecham sob interseção: exemplo

O que significa "não fechada sob interseção": existem duas LLCs cuja interseção **não é LLC**. Considere:

$$L_1 = \{a^n b^n c^m \mid n, m \geq 0\}, \quad L_2 = \{a^m b^n c^n \mid n, m \geq 0\}$$

Ambas são LLCs - cada uma força **apenas uma** igualdade de contagem (uma pilha basta).

Mas a interseção é:

$$L_1 \cap L_2 = \{a^n b^n c^n \mid n \geq 0\}$$

Essa linguagem exige **duas contagens simultâneas** (n as com n bs e n bs com n cs), e uma única pilha não basta. Usando o **Lema do Bombeamento para LLCs** (próximo slide), prova-se que $L_1 \cap L_2$ **não é LLC**.

☰ Lema do Bombeamento para LLCs (intuição)

Assim como há um LB para linguagens regulares, existe um **LB para LLCs** - com enunciado e uso *diferentes*.

Enunciado (informal): Seja L uma LLC. Existe $n \in \mathbb{N}$ tal que toda $z \in L$ com $|z| \geq n$ pode ser escrita como

$$z = uvwx y$$

satisfazendo:

1. $|vwx| \leq n$
2. $vx \neq \varepsilon$ (pelo menos uma das duas partes bombeadas é não vazia)
3. Para todo $i \geq 0$: $u v^i w x^i y \in L$

Em palavras simples: em vez de bombear **um único** pedaço (como no LB regular), agora bombeamos **dois pedaços simultaneamente** (v e x), com um "miolo" w entre eles.

Por que a diferença? LLCs vêm de árvores de derivação. Se uma árvore é alta o bastante, algum caminho da raiz à folha repete uma variável, o que cria dois trechos bombeáveis.

Uso típico: para provar que $\{a^n b^n c^n\}$ não é LLC, escolhe-se $z = a^n b^n c^n$. Não importa como se decomponha $z = uvwxy$ com $|vwx| \leq n$, bombear acabará criando uma string com contagens descompassadas.

6. APLICAÇÕES PRÁTICAS

</> GLCs em compiladores: análise sintática

A aplicação mais direta de GLCs é na **análise sintática** (*parsing*) de linguagens de programação.

Exemplo (fragmento da gramática de C para **if**):

```
statement → if_stmt | assign_stmt | while_stmt | ...
if_stmt   → 'if' '(' expr ')' statement
           | 'if' '(' expr ')' statement 'else' statement
assign_stmt → id '=' expr ';'

```

O problema do *dangling else*: a gramática acima é **ambígua**!

```
if (a) if (b) x=1; else x=2;
```

O **else** pertence ao primeiro **if** ou ao segundo? A maioria das linguagens convencionais associa ao **if mais próximo**, e a gramática precisa ser reformulada para codificar essa escolha - exatamente como fizemos com precedência de $+$ e $*$.

</> PDA na prática: parênteses balanceados

Um dos exemplos mais práticos de PDA é a verificação de **parênteses balanceados** - usada por quase todo interpretador, IDE e compilador.

Gramática:

$$S \rightarrow \varepsilon \mid SS \mid (S) \mid [S] \mid \{S\}$$

Algoritmo com pilha (simulação do PDA):

```

pilha = vazia
para cada símbolo c na string:
    se c é '(' ou '[' ou '{':
        empilhar c
    se c é ')' ou ']' ou '}':
        se pilha vazia: ERRO (fecha sem abrir)
        topo = desempilhar
        se topo não combina com c: ERRO (tipo errado)
se pilha não vazia: ERRO (abre sem fechar)
senão: OK
    
```

Exemplos: (**{[]}**) ✓ (**[]**) ✗ (**((**) ✗ **{[]}()** ✓

</> Implementação em Python: parênteses balanceados

```
def balanceado(s):
    pilha = []
    pares = {'(': ')', '[': ']', '{': '}'
    for c in s:
        if c in '([{':
            pilha.append(c)
        elif c in ')]}':
            if not pilha or pilha[-1] != pares[c]:
                return False
            pilha.pop()
    return len(pilha) == 0

# Testes
print(balanceado("({[]})")) # True
print(balanceado("([)]")) # False
print(balanceado("((()")) # False
print(balanceado("{}[](())")) # True
```

Este algoritmo [é um PDA simulado em software](#). A lista `pilha` implementa a pilha do PDA. A linguagem dos parênteses balanceados é LLC (não regular, por exigir contagem de profundidade de aninhamento).

Estruturas aninhadas: HTML, XML e JSON

Formatos com **estrutura aninhada** são naturalmente descritos por GLCs. Exemplo em XML:

```
<doc>
  <section>
    <title>Exemplo</title>
    <para>Texto aqui.</para>
  </section>
</doc>
```

GLC simplificada (ignorando atributos, texto e a regra de que as tags casam):

$$\text{Elem} \rightarrow \langle t \rangle \text{Conteudo} \langle /t \rangle \quad \text{Conteudo} \rightarrow \text{Elem Conteudo} \mid \text{texto} \mid \varepsilon$$

JSON também admite uma GLC canônica (forma compacta):

$$\text{valor} \rightarrow \text{obj} \mid \text{arr} \mid \text{string} \mid \text{num} \mid \text{bool} \mid \text{null}$$

$$\text{obj} \rightarrow \{ \} \mid \{ \text{pares} \}, \quad \text{pares} \rightarrow \text{par} \mid \text{par}, \text{pares}$$

$$\text{par} \rightarrow \text{string:valor}, \quad \text{arr} \rightarrow [] \mid [\text{itens}]$$

Nuance importante: uma GLC "pura" descreve a **sintaxe bruta** de aninhamento. Restrições como **tag de abertura casar com a de fechamento** exigem checagens adicionais no parser. Ainda assim, a parte de **aninhamento** é LLC, por isso esses parsers usam **pilha**.

7. RESUMO E EXERCÍCIOS

✓ Resumo da Aula

1. **Lema do Bombeamento (regulares):** toda linguagem regular satisfaz o LB. Usamos o contrapositivo, por contradição, para provar que certas linguagens **não são regulares**.
2. **Autômatos com Pilha (PDAs):** estendem NFAs com uma pilha de memória ilimitada. A função de transição tem assinatura $\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times \Gamma \rightarrow \mathcal{P}(Q \times \Gamma^*)$.
3. **Gramáticas Livres de Contexto (GLCs):** formalismo **gerador** equivalente aos PDAs. Descrevem linguagens por regras de produção $X \rightarrow W$ e geram árvores de derivação.
4. **Equivalência (Teorema 4.3.1):** PDAs e GLCs definem **a mesma classe** - as LLCs.
5. **Hierarquia de classes:**

Regulares $\subsetneq$ LLCs $\subsetneq$ Recursivas $\subsetneq$ Recursivamente Enumeráveis

6. **LB para LLCs:** ferramenta análoga ao LB regular, mas com **duas partes pumpáveis** simultâneas. Usado para provar que certas linguagens **não são LLCs** (ex.: $\{a^n b^n c^n\}$).

❓ Verificação Rápida

1. A linguagem $L = \{0^n 1^m \mid n \neq m\}$ é regular?
2. Um PDA é mais poderoso que um DFA? Justifique.
3. A gramática $S \rightarrow 0S1 \mid \varepsilon$ gera qual linguagem?
4. Por que L_{RR} (palíndromos pares) é reconhecida por PDA, mas **não** por DPDA?
5. A gramática $S \rightarrow SS \mid (S) \mid \varepsilon$ é ambígua? Justifique.

✓ Respostas da Verificação Rápida

1. **Não.** Argumento por **fechamento**: se L fosse regular, então $\overline{L} \cap 0^*1^*$ também seria regular (regulares fecham sob interseção e complemento). Mas $\overline{L} \cap 0^*1^* = \{0^n1^n \mid n \geq 0\}$, que **não é regular** (provado pelo LB). Contradição.
2. **Sim.** Todo DFA pode ser visto como PDA que ignora a pilha, logo $L(\text{DFA}) \subseteq L(\text{PDA})$. Além disso, PDAs aceitam $\{0^n1^n\}$, que nenhum DFA aceita. Portanto a inclusão é **estrita**.
3. $L = \{0^n1^n \mid n \geq 0\}$. Cada aplicação de $S \rightarrow 0S1$ adiciona simultaneamente um **0** à esquerda e um **1** à direita; $S \rightarrow \varepsilon$ finaliza.
4. O PDA precisa **adivinhar** onde termina a primeira metade w para iniciar o casamento com w^R . Um DPDA, por ser determinístico, não consegue adivinhar sem olhar adiante - e $\{ww^R\}$ é um exemplo clássico de LLC que **não é** determinística.
5. **Sim.** A string ε admite infinitas árvores distintas: $S \rightarrow \varepsilon$ direto; ou $S \rightarrow SS$ com ambas as cópias gerando ε ; ou $S \rightarrow SS \rightarrow SSS$ com todas gerando ε ; etc. Como existem **duas (ou mais) árvores de derivação diferentes** para a mesma string, a gramática é ambígua.

Exercícios

Exercício 6.1. Use o Lema do Bombeamento para provar que $L_{EQ} = \{w \mid w \text{ tem o mesmo número de 0s e 1s}\}$ não é regular.

Exercício 6.2. Construa um PDA que aceite $L = \{0^n 1^n \mid n \geq 1\}$. Descreva os estados, o alfabeto da pilha e as transições. Especifique o critério de aceitação.

Exercício 6.3. Forneça uma GLC para cada linguagem:

- (a) $\{a^n b^n c^m \mid n, m \geq 0\}$
- (b) $\{a^m b^n c^n \mid m, n \geq 0\}$
- (c) $\{ww^R \mid w \in \{a, b\}^*\}$ (palíndromos pares)

Exercícios (cont.)

Exercício 6.4. Prove que $L = \{a^n b^n c^n \mid n \geq 0\}$ **não é LLC** usando o Lema do Bombeamento para LLCs. *Dica: escolha $z = a^n b^n c^n$ e analise onde vwx pode estar em relação aos três blocos.*

Exercício 6.5. Considere a gramática $G = (\{S\}, \{0, 1\}, P, S)$ com regras

$$S \rightarrow \varepsilon \mid 0 \mid 1 \mid 0S0 \mid 1S1$$

- (a) Qual linguagem G descreve?
- (b) G é ambígua? Justifique.
- (c) Forneça a árvore de derivação para a string **01010**.

Exercício 6.6 (Desafio). Forneça uma GLC **não ambígua** para expressões com $+$, $*$ e parênteses, respeitando: precedência de $*$ sobre $+$, e associatividade à esquerda para ambos.

SOLUÇÕES SELECIONADAS

✔ Solução do Exercício 6.1

Prova de que L_{EQ} não é regular.

Suponha, por contradição, que L_{EQ} é regular. Pelo LB, existe t .

Escolha $w = 0^t 1^t \in L_{EQ}$ (tem t zeros e t uns, $|w| = 2t \geq t$).

Pelo LB, $w = xyz$ com $y \neq \varepsilon$ e $|xy| \leq t$. Como os primeiros t símbolos de w são todos 0, a parte xy está dentro do bloco de 0s. Portanto $y = 0^j$ com $j \geq 1$.

Tome $k = 2$: xy^2z tem $t + j$ zeros e t uns, com $j \geq 1$. Portanto xy^2z tem **mais zeros do que uns**, ou seja, $xy^2z \notin L_{EQ}$.

Isso contradiz a condição (3) do LB. Logo L_{EQ} não é regular. $\square$

✔ Solução do Exercício 6.2

PDA para $L = \{0^n 1^n \mid n \geq 1\}$ (aceitação por estado final):

$$P = (\{q_0, q_1, q_2\}, \{0, 1\}, \{0, Z_0\}, \delta, q_0, Z_0, \{q_2\})$$

Transições (convenção: 1º símbolo empilhado fica no topo):

De	Para	Lê	Topo	Empilha	Efeito
q_0	q_0	0	Z_0	$0 Z_0$	empilha primeiro 0
q_0	q_0	0	0	00	empilha outro 0
q_0	q_1	1	0	ϵ	lê 1º 1, desempilha
q_1	q_1	1	0	ϵ	casa cada 1 com um 0
q_1	q_2	ϵ	Z_0	Z_0	aceita (pilha no fundo)

Funcionamento: q_0 empilha um 0 para cada 0 lido; q_1 desempilha um 0 para cada 1 lido. A transição final para q_2 só ocorre quando o topo é Z_0 - o que significa que todos os 0s empilhados foram casados com exatamente o mesmo número de 1s.

✓ Solução do Exercício 6.3 (esboço)

(a) $\{a^n b^n c^m \mid n, m \geq 0\}$ - uma variável para o bloco balanceado, outra para os c s livres:

$$S \rightarrow XC \quad X \rightarrow aXb \mid \varepsilon \quad C \rightarrow cC \mid \varepsilon$$

(b) $\{a^m b^n c^n \mid m, n \geq 0\}$ - simétrico ao anterior:

$$S \rightarrow AY \quad A \rightarrow aA \mid \varepsilon \quad Y \rightarrow bYc \mid \varepsilon$$

(c) $\{ww^R \mid w \in \{a, b\}^*\}$ - envelopamento recursivo, análogo a G_{pal} :

$$S \rightarrow aSa \mid bSb \mid \varepsilon$$

Observação: as gramáticas (a) e (b) mostram que cada parte individualmente precisa apenas de **uma contagem balanceada**. É por isso que são LLCs. A interseção delas, como vimos, exige **duas contagens** simultâneas e escapa do poder expressivo das LLCs.